

Software Project Lab 1

Name:Fahmida Munni
BSSE Roll:1604

CodeJudge

Automated Testing and Quality Analysis Tool for C
Programs

Supervised by,

Md. Saeed Siddik
Assistant Professor
IIT, University of Dhaka

Project Overview

Codejudge is a tool for evaluating C programs. It evaluates C programs by running it(C file) against test cases through a command-line interface. Beyond checking if code works correctly, it also provides metrics on code quality and complexity.

Core Components

Modules

- **Compilation Engine:**
GCC integration, error capture
- **Execution Controller:**
Process isolation, timeout enforcement, signal monitoring
- **Validation Engine:**
Output comparison
- **Analysis Module:**
Test aggregation, metrics calculation

Features Implemented So Far

1. Error Detection & Handling
2. Automated Testing System
3. Code Metrics Analysis

```
===== MENU =====  
1. Run Test Case  
2. View Test Results  
3. Analyze Code Metrics  
4. Exit  
=====
```


Features Implemented So Far

Compilation Error Detection

```
int compile_program(const char *source_file) {
    snprintf(command, cmd_size,
              "gcc %s -o myprogram -lm 2> error.txt",
              source_file);

    int compile_result = system(command);

    FILE *error_file = fopen("error.txt", "r");
    if (error_file != NULL) {
        // Display compilation errors
        while ((ch = fgetc(error_file)) != EOF) {
            putchar(ch);
            has_error = 1;
        }
        if (has_error) return 0;
    }
    return 1;
}
```

Features Implemented So Far

Compilation Error Detection

Input File:

```
#include <stdio.h>

int main()
{
    int x = 5
    printf("%d", x);
    return 0;
}
```

Output:

```
Enter C file path: test/ce.c
Compiling test/ce.c...
Compilation output:
test/ce.c: In function 'main':
test/ce.c:6:5: error: expected ',' or ';' before 'printf'
      6 |     printf("%d", x);
        |         ^~~~~~
Compilation FAILED
```

Key Features Implemented

Runtime Error Detection

```
void run_program_and_Capture_the_Output(DBuffer *buffer) {  
    int exit_code = system("timeout 10s ./myprogram < my_input.txt > my_output.txt 2>&1");  
  
    if (WIFEXITED(exit_code)) {  
        int status = WEXITSTATUS(exit_code);  
        if (status == 124) {  
            append_to_buffer(buffer, "\n[TIMEOUT: Program exceeded 10 seconds]");  
        }  
    } else if (WIFSIGNALED(exit_code)) {  
        int signal_num = WTERMSIG(exit_code);  
        if (signal_num == SIGSEGV) {  
            append_to_buffer(buffer, "\n[RUNTIME ERROR: Segmentation fault]");  
        } else if (signal_num == SIGFPE) {  
            append_to_buffer(buffer, "\n[RUNTIME ERROR: Floating point exception]");  
        }  
    }  
}
```


Key Features Implemented

Runtime Error Detection

Input file

```
1 // 10 seconds ago | Author: (100)  
2 #include <stdio.h>  
  
3 int main() {  
4     while(1)  
5     {  
6         //infinite loop  
7     }  
8     return 0;  
9 }
```

Output

Actual Output:

```
[TIMEOUT: Program exceeded 10 seconds]
```

```
=====
```

```
TIMEOUT - Possible infinite loop
```

Key Features Implemented

Test Result

```
typedef struct {  
    int test_number;  
    int passed;  
    char *input;  
    char *expected;  
    char *actual;  
    char *error_msg;  
} TestResult;
```

Key Features Implemented

LOC & Halstead metrics

```
typedef struct {  
    int loc;  
    int lloc;  
    int comments;  
    int halstead_n1;  
    int halstead_n2;  
    int halstead_N1;  
    int halstead_N2;  
    double halstead_length;  
    double halstead_vocab;  
    double halstead_volume;  
    double halstead_difficulty;  
    double halstead_effort;  
    int functions;  
} CodeMetrics;
```

Future Enhancements

1. Integration of Cyclomatic Metrics
2. Integration of Cognitive Metrics
3. CSV Export for Test Case Visualization

Github:<https://github.com/innu-M/CodeJudge-SPL1-.git>

Thank You